Batch: C3 Roll No.: 16010123325

Experiment / Assignment / Tutorial No. 5

Title: Implementation of DBScan clustering Algorithm

Aim: To understand & implement DBScan clustering Algorithm using python.

Expected Outcome of Experiment:

CO4: Apply clustering and dimension reduction algorithm to analyse and interpret data patterns effectively.

Books/ Journals/ Websites referred:

1. "Machine Learning", Tom M. Mitchell, McGraw Hill, 2017.
2. https://www.geeksforgeeks.org/machine-learning/decision-tree-introduction-example/
3. https://medium.com/@abhishekjainindore24/all-about-decision-trees-80ea55e37fef

Theory of DBScan Clustering Algorithm:

DBSCAN is a density-based clustering algorithm that groups data points that are closely packed together and marks outliers as noise based on their density in the feature space. It identifies clusters as dense regions in the data space separated by areas of lower density. Unlike K-Means or hierarchical clustering which assumes clusters are compact and spherical, DBSCAN perform well in handling real-world data irregularities such as:

- Arbitrary-Shaped Clusters: Clusters can take any shape not just circular or convex.
- Noise and Outliers: It effectively identifies and handles noise points without assigning them to any cluster.

Key Parameters in DBSCAN:

1. eps: This defines the radius of the neighborhood around a data point. If the distance between two points is less than or equal to eps they are considered neighbors. A

common method to determine eps is by analyzing the k-distance graph. Choosing the right eps is important:

- If eps is too small most points will be classified as noise.
- If eps is too large clusters may merge and the algorithm may fail to distinguish between them.

2. MinPts: This is the minimum number of points required within the eps radius to form a dense region. A general rule of thumb is to set MinPts >= D+1 where D is the number of dimensions in the dataset.

DBSCAN works by categorizing data points into three types:

1. Core points which have a sufficient number of neighbors within a specified radius (eplison)
2. Border points which are near core points but lack enough neighbors to be core points themselves
3. Noise points which do not belong to any cluster.

Steps in the DBSCAN Algorithm:

1. Identify Core Points: For each point in the dataset count the number of points within its eps neighborhood. If the count meets or exceeds MinPts mark the point as a core point.
2. Form Clusters: For each core point that is not already assigned to a cluster create a new cluster. Recursively find all density-connected points i.e points within the eps radius of the core point and add them to the cluster.
3. Density Connectivity: Two points a and b are density-connected if there exists a chain of points where each point is within the eps radius of the next and at least one point in the chain is a core point. This chaining process ensures that all points in a cluster are connected through a series of dense regions.
4. Label Noise Points: After processing all points any point that does not belong to a cluster is labeled as noise.

Evaluation Metrics For DBSCAN Algorithm In Machine Learning :

Use the Silhouette score and Adjusted rand score for evaluating clustering algorithms.

- Silhouette's score is in the range of -1 to 1. A score near 1 denotes the best meaning that the data point i is very compact within the cluster to which it belongs

and far away from the other clusters. The worst value is -1. Values near 0 denote overlapping clusters.

- Absolute Rand Score is in the range of 0 to 1. More than 0.9 denotes excellent cluster recovery and above 0.8 is a good recovery. Less than 0.5 is considered to be poor recovery.

Problem Definition:

Apply DBScan Clustering algorithm and evaluate its performance using different metrics.

Details of data set used:

The dataset used is the **Mall Customers Dataset**, which contains information about customers visiting a shopping mall. It includes a total of **200 records**.

For clustering, relevant features such as **Gender, Age, Annual Income, and Spending Score** were used. The data was **standardized** before applying DBSCAN to ensure fair distance computation.

API/Function used:

With Python Libraries

- pandas.read_csv() → Load dataset
- map() → Convert gender to numeric
- StandardScaler() → Normalize features
- DBSCAN() → Perform clustering
- fit_predict() → Generate cluster labels
- silhouette_score() → Evaluate clustering performance

Without Library

- euclidean_distance() → Compute distance between points
- region_query() → Find neighbors within ε radius
- expand_cluster() → Expand clusters from core points
- dbscan() → Main DBSCAN algorithm logic

Implemetation:
Manual-

```
import pandas as pd
import numpy as np
from sklearn.preprocessing import StandardScaler
from sklearn.metrics import silhouette_score

# Load dataset
df = pd.read_csv('Mall_Customers.csv')

# Handle Gender
df['Gender'] = df['Gender'].map({'Male': 0, 'Female': 1})

# Select features
X = df[['Gender','Age','Annual Income (k$)','Spending Score (1-100)']].values

# Standardize
scaler = StandardScaler()
X_scaled = scaler.fit_transform(X)

# Distance function
def euclidean(p, q):
    return np.sqrt(np.sum((p - q) ** 2))

# Find neighbors
def get_neighbors(X, idx, eps):
    neighbors = []
    for i in range(len(X)):
        if euclidean(X[idx], X[i]) <= eps:
            neighbors.append(i)
    return neighbors

# Expand cluster
def expand_cluster(X, labels, idx, neighbors, cluster_id, eps, min_pts):
    labels[idx] = cluster_id
    i = 0

    while i < len(neighbors):
        n = neighbors[i]

        if labels[n] == -1:
            labels[n] = cluster_id

        elif labels[n] == 0:
            labels[n] = cluster_id
```

```
            new_neighbors = get_neighbors(X, n, eps)

            if len(new_neighbors) >= min_pts:
                neighbors += new_neighbors

        i += 1

# Manual DBSCAN
def dbscan(X, eps, min_pts):
    labels = [0] * len(X)
    cluster_id = 0

    for i in range(len(X)):
        if labels[i] != 0:
            continue

        neighbors = get_neighbors(X, i, eps)

        if len(neighbors) < min_pts:
            labels[i] = -1  # noise
        else:
            cluster_id += 1
            expand_cluster(X, labels, i, neighbors, cluster_id, eps, min_pts)

    return np.array(labels)

# Run manual DBSCAN
eps = 0.5
min_pts = 5

manual_labels = dbscan(X_scaled, eps, min_pts)
df['Manual_Cluster'] = manual_labels

# Silhouette score
mask = manual_labels != -1
if len(set(manual_labels[mask])) > 1:
    score = silhouette_score(X_scaled[mask], manual_labels[mask])
else:
    score = -1

# Output
print("Epsilon (ε):", eps)
print("Uniform Radius:", eps)
print("Min Samples:", min_pts)
print("Silhouette Score:", score)
```

```
print(df.head())
```

In-built-

```
import pandas as pd
import numpy as np
from sklearn.cluster import DBSCAN
from sklearn.preprocessing import StandardScaler
from sklearn.metrics import silhouette_score, pairwise_distances

# Load dataset
df = pd.read_csv("Mall_Customers.csv")

print("Dataset Loaded:")
print(df.head())

# Select features
X = df[['Annual Income (k$)', 'Spending Score (1-100)']]

# Scale
scaler = StandardScaler()
X_scaled = scaler.fit_transform(X)

# Distance matrix
dist_matrix = pairwise_distances(X_scaled)
print("\nDistance Matrix (first 5x5):")
print(dist_matrix[:5, :5])

# Parameters
eps = 0.5
min_samples = 5

print("\nEpsilon (eps):", eps)
print("Min Samples:", min_samples)
print("Uniform radius used for all points")

# DBSCAN
db = DBSCAN(eps=eps, min_samples=min_samples)
labels = db.fit_predict(X_scaled)

df['Cluster'] = labels

# Core points
core_indices = db.core_sample_indices_
core_mask = np.zeros_like(labels, dtype=bool)
core_mask[core_indices] = True
```

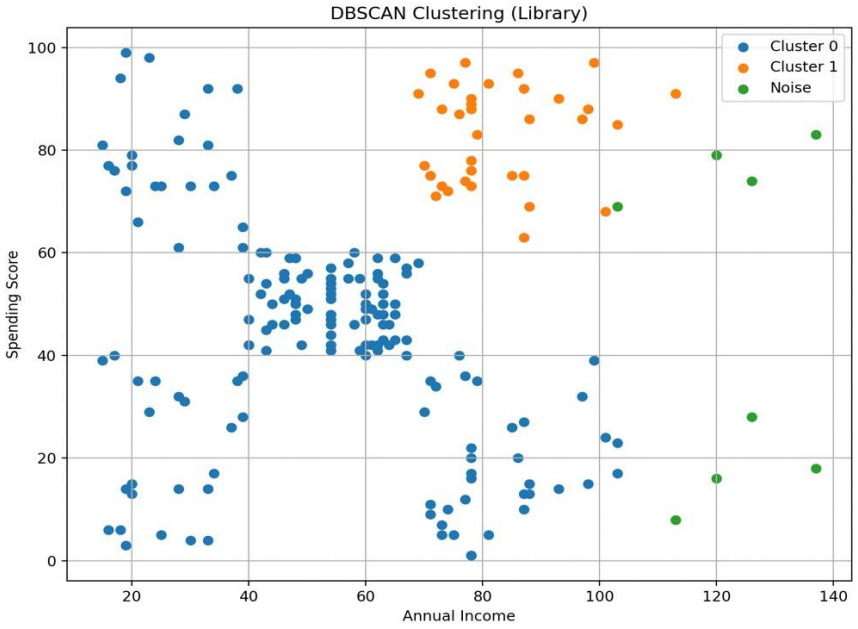
```
print("\nCore Points:", len(core_indices))
print("Noise Points:", np.sum(labels == -1))
print("Clusters Found:", set(labels))

# Silhouette
mask = labels != -1
if len(set(labels[mask])) > 1:
    score = silhouette_score(X_scaled[mask], labels[mask])
    print("\nSilhouette Score:", score)
else:
    print("\nSilhouette Score cannot be computed")

# Cluster Interpretation
print("\nCluster Interpretation:")

for cluster in sorted(df['Cluster'].unique()):
    data = df[df['Cluster'] == cluster]

    if cluster == -1:
        print(f"\nCluster {cluster} (Noise)")
        print("Count:", len(data))
    else:
        print(f"\nCluster {cluster}")
        print("Count:", len(data))
        print("Avg Income:", round(data['Annual Income (k$)'].mean(), 2))
        print("Avg Spending:", round(data['Spending Score (1-100)'].mean(),
2))

import matplotlib.pyplot as plt

plt.figure(figsize=(8,6))

for label in set(labels):
    if label == -1:
        name = "Noise"
        color = 'black'
    else:
        name = f"Cluster {label}"
        color = None

    subset = df[df['Cluster'] == label]

    plt.scatter(subset['Annual Income (k$)'],
                subset['Spending Score (1-100)'],
```

```
                label=name)

plt.xlabel("Annual Income")
plt.ylabel("Spending Score")
plt.title("DBSCAN Clustering")
plt.legend()
plt.grid(True)
plt.show()
```

Results:

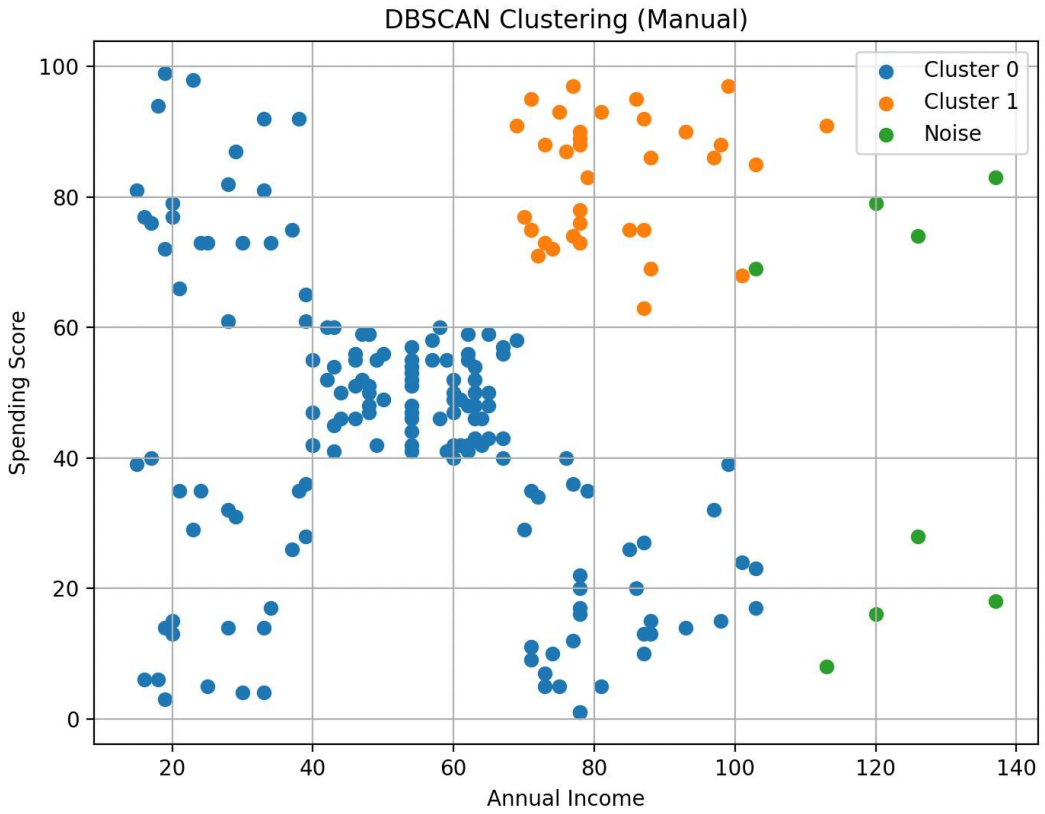
With Python Libraries

```
shreya@Shreyas-MacBook-Air-2 ML % python3 db_lib.py
Dataset Loaded:
   CustomerID  Gender  Age  Annual Income (k$)  Spending Score (1-100)
0           1    Male   19                  15                      39
1           2    Male   21                  15                      81
2           3  Female   20                  16                       6
3           4  Female   23                  16                      77
4           5  Female   31                  17                      40

Distance Matrix (first 5x5):
[[0.         1.63050555 1.28167999 1.47571302 0.08564307]
 [1.63050555 0.         2.91186723 0.15990848 1.59351358]
 [1.28167999 2.91186723 0.         2.75633081 1.32048483]
 [1.47571302 0.15990848 2.75633081 0.         1.4369048 ]
 [0.08564307 1.59351358 1.32048483 1.4369048  0.        ]]

Epsilon (eps): 0.5
Min Samples: 5
Uniform radius used for all points

Core Points: 185
Noise Points: 8
Clusters Found: {np.int64(0), np.int64(1), np.int64(-1)}

Silhouette Score: 0.3875583892728279

Cluster Interpretation:

Cluster -1 (Noise)
Count: 8

Cluster 0
Count: 157
Avg Income: 52.49
Avg Spending: 43.1

Cluster 1
Count: 35
Avg Income: 82.54
Avg Spending: 82.8
```



Without Python Libraries

```
shreya@Shreyas-MacBook-Air-2 ML % python3 db.py
Dataset Loaded:
   CustomerID  Gender  Age  Annual Income (k$)  Spending Score (1-100)
0           1    Male   19                  15                      39
1           2    Male   21                  15                      81
2           3  Female   20                  16                       6
3           4  Female   23                  16                      77
4           5  Female   31                  17                      40

Distance Matrix (first 5x5):
[[0.         1.63050555 1.28167999 1.47571302 0.08564307]
 [1.63050555 0.         2.91186723 0.15990848 1.59351358]
 [1.28167999 2.91186723 0.         2.75633081 1.32048483]
 [1.47571302 0.15990848 2.75633081 0.         1.4369048 ]
 [0.08564307 1.59351358 1.32048483 1.4369048  0.        ]]

Epsilon (eps): 0.5
Min Samples: 5
Uniform radius used for all points

Core Points: Approx (manual)
Noise Points: 8
Clusters Found: {np.int64(0), np.int64(1), np.int64(-1)}

Silhouette Score: 0.3875583892728279

Cluster Interpretation:

Cluster -1 (Noise)
Count: 8

Cluster 0
Count: 157
Avg Income: 52.49
Avg Spending: 43.1

Cluster 1
Count: 35
Avg Income: 82.54
Avg Spending: 82.8
```



Interpretation:

- **Cluster -1 (Noise)**

Contains customers that do not belong to any dense region. These represent **outliers**, typically having unusual combinations of income and spending behavior.

- **Cluster 0**

Consists of customers with **moderate income and average spending score**. This group represents the **general customer segment** with balanced purchasing behavior.

- **Cluster 1**

Includes customers with **high income and high spending score**, indicating **premium or high-value customers** who contribute significantly to revenue.

Conclusion:

DBSCAN was successfully applied to cluster customers based on their behavior, effectively identifying natural groupings as well as outliers without predefining the number of clusters. The results showed moderate clustering quality and consistent outcomes across both manual and library implementations.

Post Lab :

1. List and explain the various distance metrics used to measure the similarity.

Distance metrics are used to calculate how similar or dissimilar two data points are.

- **Euclidean Distance**:
  Most commonly used; calculates straight-line distance between two points.
  Suitable for continuous numerical data.
- **Manhattan Distance**:
  Measures distance as the sum of absolute differences along each dimension.
  Useful when movement is grid-like (city block distance).
- **Minkowski Distance**:
  Generalized form of Euclidean and Manhattan distance.
  Controlled by a parameter p (p=1 → Manhattan, p=2 → Euclidean).
- **Cosine Similarity**:
  Measures angle between two vectors rather than distance.
  Used when direction matters more than magnitude (e.g., text data).

2. Evaluate any two other Evaluation Indexes for the evaluation of clustering algorithms.

Davies-Bouldin Index (DB Index)

- Measures **cluster compactness and separation**
- Lower value → better clustering
- Based on ratio of intra-cluster distance to inter-cluster distance

Calinski-Harabasz Index (CH Index)

- Also called **Variance Ratio Criterion**
- Higher value → better clustering
- Ratio of between-cluster variance to within-cluster variance